

The background of the slide features a large, faint, light-blue circular logo of Tianjin University in the upper right corner. The logo contains the university's name in English ('TIANJIN UNIVERSITY') and Chinese ('天津大学'), along with the founding year '1895'. In the lower left corner, there is a faint, light-blue line drawing of a traditional Chinese building with a tiled roof and multiple windows.

内存管理

Memory Management



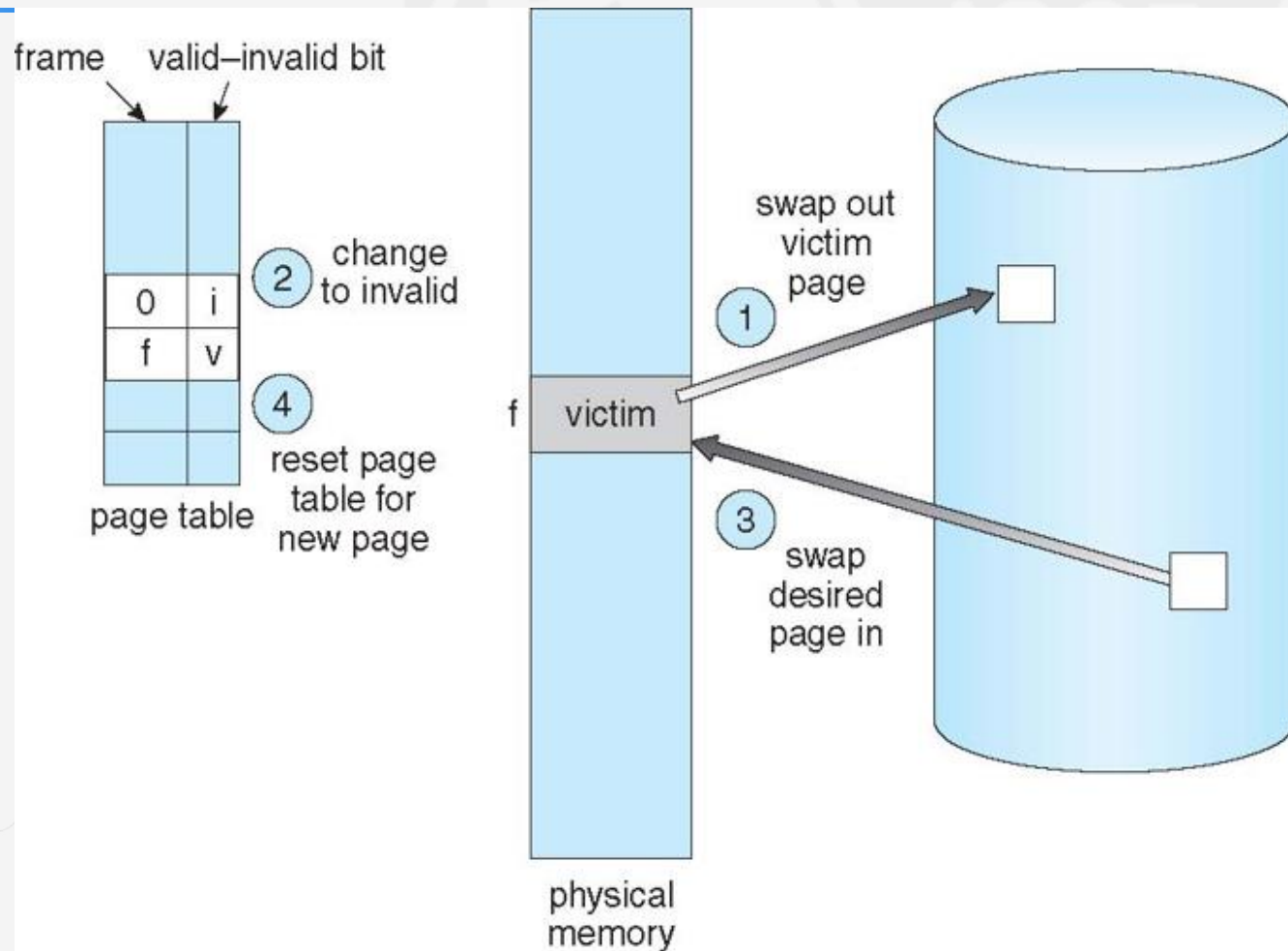
主要内容

- 页面置换
- 页面置换算法：OPT、NRU、FIFO、SC、CLOCK、LRU、Aging、LFU等
- 置换策略（全局/局部）



页面置换 (Page Replacement)

- 为了为进程提供比实际大的多的虚拟内存空间，MMU会按需装入用到的页面。即当要访问的页面存在位P位（或V位）为0的时，引发缺页中断，MMU将页面装入内存并更新页表和TLB。
- 当内存空闲空间低于某个阈值时，MMU会启动页面置换程序，淘汰掉一些暂时用不到的页面到辅存中，以保证内存中有足够的空闲空间，以便及时应对未来缺页中断处理程序的需求。





页面置换算法 (Page Replacement Algorithms)

- 页面置换时，一个非常重要的问题，就是选择什么样的页面进行淘汰，这就是页面置换算法要决定的
- 常见的页面置换算法有：
- 最优置换算法：OPT - Optimal page replacement algorithm
- 最近未使用：NRU - Not recently used
- 先进先出：FIFO - First-In, First-Out
- 第二次机会：SC - Second chance
- 时钟：Clock algorithm
- 最近最久未使用：LRU - Least recently used

- 老化算法：Aging algorithm



OPT - Optimal Page Replacement

- **最优置换(OPT)算法**选择的被淘汰页面，将是以后永不使用的，或许是在最长(未来)时间内不再被访问的页面。采用最优置换算法可保证获得最低的缺页率。但是由于无法预知哪一个页面是未来最长时间不再被访问的，因而该算法是无法实现的。这个算法通常用来作为基准来衡量其他算法的优劣程度。
- 例：在一个请求分页系统中，某个作业的页面走向为：1, 3, 2, 1, 1, 3, 4, 1, 3, 2, 1, 4. 当分配给该作业的物理页帧为3时，采用OPT算法，计算访问过程中所发生的缺页率为多少？(5/12)

1	3	2	1	1	3	4	1	3	2	1	4
1	3	2				4			4		
	1	3				3			2		
		1				1			1		
*	*	*				*			*		



最近未使用页面置换算法 (NRU)

- 系统为每一个页面设置两个标志位：当页面被访问时设置R位为1，当页面（修改）被写入时设置M位为1，每个时钟滴答结束后将所有页面的R位重置为0。
- 当发生缺页中断时，MMU检查所有的页面，并根据它们当前的R和M位的值，分为四种情况。越靠上的越被优先换出。即在最近的一个时钟滴答内，淘汰一个没有被访问但是已经被修改的页面，比淘汰一个被频繁使用但是“clean”的页面要好。
- NRU算法虽然非常简陋，但因其实现起来简单，在实际的操作系统系统中经常被采用。

R (referenced)	M (modified)
0	0
0	1
1	0
1	1



FIFO - First-In, First-Out page replacement

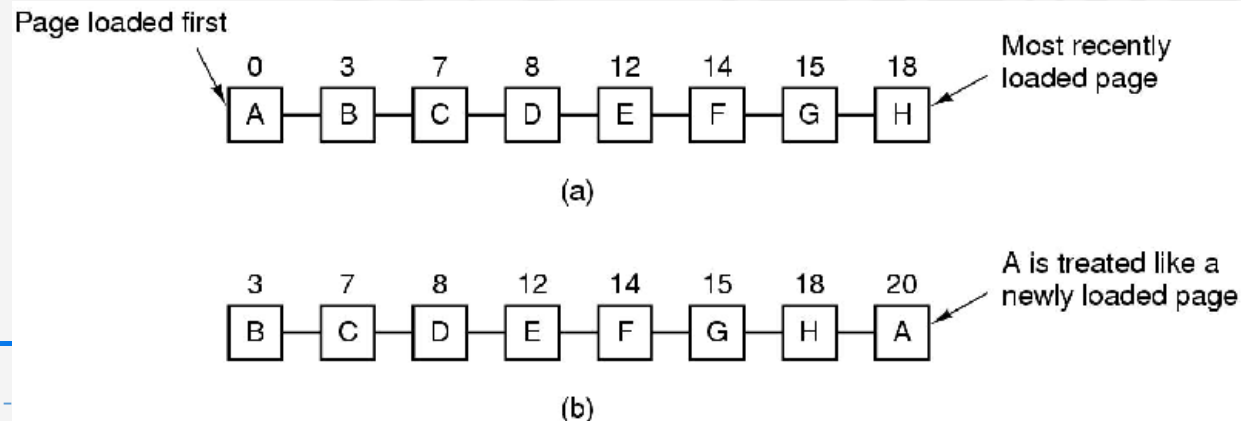
- **先进先出（FIFO）算法**淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面予以淘汰。按照进入内存的先后次序排列成队列，从队尾进入，从队首删除。但是该算法会淘汰经常访问的页面，不适应进程实际运行的规律，目前已经很少使用。
- 例：在一个请求分页系统中，某个作业的页面走向为：1，3，2，1，1，3，4，1，3，2，1，4。当分配给该作业的物理页帧为3时，采用FIFO算法，计算访问过程中所发生的缺页率为多少？（8/12）

1	3	2	1	1	3	4	1	3	2	1	4
1	3	2				4	1	3	2		4
	1	3				2	4	1	3		2
		1				3	2	4	1		3
*	*	*				*	*	*	*		*

1. 插入新页面时会插入到链表头部；
2. 当链表满的时候，将链表尾部的页面淘汰。



SC - Second chance page replacement



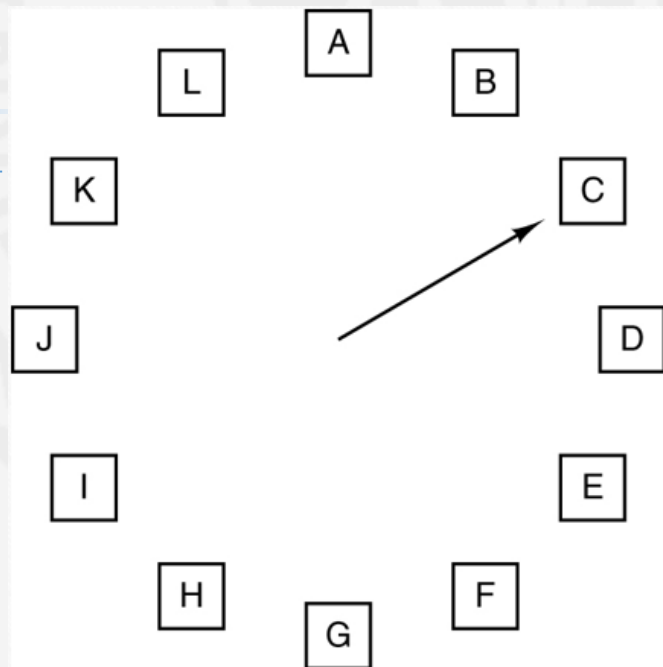
- **第二次机会 (SC) 算法**是对FIFO的一种改进。在使用FIFO更换一个页面时，需要看一下该页面是否在最近被访问过，如果没有被访问过，则替换该页面。反之，如果最近被访问过（通过检查其访问位的取值），则不替换该页面，而是将该页面挂到链表末端，并将该页面进入内存的时间设置为当前时间，并将其访问位清零。这样，对于最近被访问过的页面来说，相当于给了它第二次机会。
- 例：页面A-H依照FIFO排队上面数字表示到达时间。
- 1. 淘汰从最先到达的页面A开始。如果页面A的R位为0，说明最近没有被访问过，直接替换。
- 2. 如果页面A的R位为1，说明A最近被访问过，保留页面A但是设置A的R位为0，并将A的到达时间重置为当前时间20，挂到链表末端。
- 3. 下一个处理的页面就是B，依此类推。

- 4. 极端情况下所有页面的R位都为1。队伍重排后R位都为0。此时淘汰页面A。



clock page replacement

- 时钟（clock）算法与第二次机会（SC）算法完全一样
- 只是用环形队列（形状像时钟）代替了SC中的普通队列。
- 环形队列依照页面装入时间排列，指针指向最早装入的页面
- 需要淘汰页面时，检查指针所指页面
 - 如果该页面 $R=0$ ，说明从上次检查到这次，该页面没有被访问过，直接淘汰
 - 如果该页面 $R=1$ ，重置其 R 位为0，将指针顺时针转动到下一个页面
 - 重复这些步骤，直到找到一个访问位为0的页面。





最近最久未使用置换算法（Least Recently Used, LRU）

■ **最近最久未使用置换算法（Least Recently Used, LRU）** 置换最近一段时间以来最长时间未访问过的页面。根据程序局部性原理，刚被访问的页面，可能马上又要被访问；而较长时间内没有被访问的页面，可能最近不会被访问。

■ 例：在一个请求分页系统中，某个作业的页面走向为：1，3，2，1，1，3，4，1，3，2，1，4.当分配给该作业的物理页帧为3时，采用LRU算法，计算访问过程中所发生的缺页率为多少？（6/12）

1	3	2	1	1	3	4	1	3	2	1	4
1	3	2	1		3	4	1	3	2	1	4
	1	3	2		1	3	4	1	3	2	1
		1	3		2	1	3	4	1	3	2
*	*	*				*			*		*

1. 插入新页面时会插入到链表头部；
2. 当内存中页面被访问时，将该页面移到链表头部；
3. 当链表满的时候，将链表尾部的页面淘汰。



LRU的近似实现

■ LRU算法普遍地适用于各种类型的程序，但是系统要时时刻刻对各页的访问历史情况加以记录和更新，开销太大，因此LRU算法必须要有硬件的支持。

■ 例：引用串 0, 1, 2, 3, 2, 1, 0, 3, 2, 3.

■ 需要在cache中有一个4*4的计数器。访问某一页时，相应行全置1，然后相应列全置0。最后淘汰1最少的行对应页。

■ 实际系统中因页面太多，无法实现。

	Page			
	0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

(a)

	Page			
	0	1	2	3
0	0	0	1	1
1	1	0	1	1
2	0	0	0	0
3	0	0	0	0

(b)

	Page			
	0	1	2	3
0	0	0	0	1
1	1	0	0	1
2	1	1	0	1
3	0	0	0	0

(c)

	Page			
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0

(d)

	Page			
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	1
3	1	1	0	0

(e)

0	0	0	0
1	0	1	1
1	0	0	1
1	0	0	0

(f)

0	1	1	1
0	0	1	1
0	0	0	1
0	0	0	0

(g)

0	1	1	0
0	0	1	0
0	0	0	0
1	1	1	0

(h)

0	1	0	0
0	0	0	0
1	1	0	1
1	1	0	0

(i)

0	1	0	0
0	0	0	0
1	1	0	0
1	1	1	0

(j)



老化算法 (Aging)

- 另一种优秀的 LRU 近似算法为**老化算法 (Aging)**，可以不使用额外硬件支持。
- 每个页面有一个8位的计数器。前一个时钟滴答 (clock tick) 结束时，所有页面的R位均置0。一个时钟滴答当中，若页面被访问到，则R位被置为1。时钟滴答结束时，每个页面的计数器右移一位，将R位记录在空出来的最高位上。
- 页面置换时，淘汰计数器值最小的页面。本例中为页面3。

	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000
	(a)	(b)	(c)	(d)	(e)



最不常用算法 (LFU)

- **最不常用算法 (LFU - Least Frequently Used)**。根据数据的历史访问频率来淘汰数据。理论是假设访问次数越多的页面，很有可能在未来被再次访问。
- 缺点：可能会积累不再使用的数据。
 - 某些数据短时间内被重复引用，并且在很长一段时间内不再被访问。由于它的访问频率计数急剧增加，即使它在相当长的一段时间内不会被再次使用，也不会短时间内被淘汰。这使得其他可能更频繁使用的块更容易被清除。
 - 此外，刚进入缓存的新项可能很快就会再次被删除，因为它们的计数器较低，即使之后可能会频繁使用。





Belady异常

- 一般来说，分配给进程的物理块越多，运行时的缺页次数应该越少。但是，在使用FIFO时，可能存在相反情况，分配4个物理块的缺页竟然比3个物理块的缺页次数还多！
- 例如：进程访问顺序为0、2、1、3、0、2、4、0、2、1、3、4。
- M=3时，缺页中断9次。缺页中断率 $9/12 \times 100\% = 75\%$ 。

P	0	2	1	3	0	2	4	0	2	1	3	4
M=3	0	2	1	3	0	2	4	4	4	1	3	3
		0	2	1	3	0	2	2	2	4	1	1
			0	2	1	3	0	0	0	2	4	4
F=9	Y	Y	Y	Y	Y	Y	Y			Y	Y	

P	0	2	1	3	0	2	4	0	2	1	3	4
M=4	0	2	1	3	3	3	4	0	2	1	3	4
		0	2	1	1	1	3	4	0	2	1	3
			0	2	2	2	1	3	4	0	2	1
				0	0	0	2	1	3	4	0	2
F=10	Y	Y	Y	Y			Y	Y	Y	Y	Y	Y

局部/全局置换策略

- 局部置换策略 (Local Allocation Policies) : 新换入的页面只能使用分配给本进程的空闲页帧。(A6替换A5)
- 全局置换策略 (Global Allocation Policies) : 新换入的页面可以使用系统中所有空闲的页帧。(A6替换B3)

	Age
A0	10
A1	7
A2	5
A3	4
A4	6
A5	3
B0	9
B1	4
B2	6
B3	2
B4	5
B5	6
B6	12
C1	3
C2	5
C3	6

(a)

A0
A1
A2
A3
A4
A6
B0
B1
B2
B3
B4
B5
B6
C1
C2
C3

(b)

A0
A1
A2
A3
A4
A5
B0
B1
B2
A6
B4
B5
B6
C1
C2
C3

(c)



例题

页号	页框 (Page Frame) 号	有效位 (存在位)
0	101H	1
1	-----	0
2	254H	1

- 请求分页管理系统中，假设某进程的页表内容如表中所示。
- 页面大小为4KB，一次内存的访问时间是 100ns，一次快表(TLB)的访问时间是 10ns，处理一次缺页的平均时间 10^8 ns(已含更新 TLB 和页表的时间)，进程的驻留集大小固定为 2，采用最近最少使用置换算法(LRU)和局部淘汰策略。假设 (1) TLB 初始为空; (2) 地址转换时先访问 TLB，若 TLB 未命中，再访问页表(忽略访问页表之后的 TLB 更新时间); (3) 有效位为 0 表示页面不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H，请问：
 - 1) 依次访问上述三个虚地址，各需多少时间?给出计算过程。
 - 2) 基于上述访问序列，虚地址1565H 的物理地址是多少?请说明理由。



例题（解答）

页号	页框 (Page Frame) 号	有效位 (存在位)
0	101H	1
1	-----	0
2	254H	1

(1) 根据页式管理的工作原理，应先考虑页面大小，以便将页号和页内位移分解出来。页面大小为 4KB，即 2^{12} ，则得到页内位移占虚地址的低 12 位，页号占剩余高位。可得三个虚地址的页号 P 如下 (十六进制的一位数字转换成 4 位二进制，因此，十六进制的低三位正好为页内位移，最高位为页号):

>>> **2362H** : P=2, 访问快表 10ns, 因初始为空, 访问页表 100ns 得到页框号, 合成物理地址后访问 主存 100ns, 共计 10ns+100ns+100ns=**210ns**。

>>> **1565H** : P=1, 访问快表 10ns, 落空, 访问页表 100ns 落空, 进行缺页中断处理 10^8 ns, 访问快表 10ns, 合成物理地址后访问主存 100ns, 共计 10ns+100ns+ 10^8 ns+10ns+100ns=**100 000 220ns**。

>>> **25A5H** : P=2, 访问快表, 因第一次访问已将该页号放入快表, 因此花费 10ns 便可合成物理地址, 访问主存 100ns, 共计 10ns+100ns=**110ns**。

(2) 当访问虚地址 1565H 时, 产生缺页中断, 合法驻留集为 2, 必须从页表中淘汰一个页面, 根据题目的置

换算法, 应淘汰 0 号页面, 因此 1565H 的对应页框号为 101H。由此可得 1565H 的物理地址为 **101565H**。



课后阅读

- 《操作系统概念 OSC（第9版）》第8章、第9章。
- 《现代操作系统 MOS（第4版）》第3章。
- 《深入理解计算机系统 CSAPP（第3版）》第6章、第9章。